

Day 1 Internship Report

Natural Language Processing (NLP) Preprocessing Pipeline

Submitted By:

Manahil Bashir

BS Computer Science

FAST – National University of Computer and Emerging Sciences

Internship: NLP Internship

Day: 1

1. Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling computers to understand and process human language. Before text can be used for machine learning or language models, it must first go through several preprocessing steps to remove noise and convert it into a structured format.

The objective of Day 1 was to understand and execute the basic stages of an NLP preprocessing pipeline using Python.

2. Objectives

The objectives assigned for Day 1 were:

- Create an NLP preprocessing pipeline.
- Perform text cleaning.
- Compare different tokenization methods.
- Apply stemming and lemmatization.
- Perform Part-of-Speech (POS) tagging.
- Generate Bag-of-Words (BoW) representations.
- Observe the advantages and limitations of different preprocessing techniques.

3. Dataset

The provided corpus contained paragraphs from three different domains:

- News
- Science

- Dialogue

The dataset was stored in:

data/input_corpus.txt

This file served as the input for all preprocessing tasks completed during Day 1.

4. Task 1 – Text Cleaning

The first step was cleaning the raw corpus before applying any NLP techniques.

The cleaning process involved removing unnecessary symbols and normalizing the text while preserving the main content.

The cleaned output was saved as:

outputs/cleaned_corpus.txt

Observation

The cleaning script successfully produced a normalized version of the corpus. Some scientific terms containing special characters (for example, *SARS-CoV-2*) became simplified after punctuation removal. This behavior is expected when general-purpose text cleaning rules are applied.

5. Task 2 – Tokenization Comparison

The next task was to compare four different tokenization methods.

The following tokenizers were used:

- Character-level Tokenization
- Word-level Tokenization
- Byte Pair Encoding (BPE)
- WordPiece

The comparison demonstrated how each tokenizer splits the same sentence into different units.

The generated output was stored in:

outputs/tokenization_comparison.txt

Observation

Character-level tokenization produced the highest number of tokens because every character was treated as a separate unit.

Word-level tokenization preserved complete words and was easy to understand.

BPE and WordPiece divided unfamiliar words into meaningful subword units. These approaches are commonly used in modern transformer-based language models because they handle unseen words more effectively.

6. Task 3 – Stemming and Lemmatization

The third task involved comparing stemming and lemmatization.

Both techniques reduce words to simpler forms, but they behave differently.

Some examples from the output are shown below.

Original Word	Stem	Lemma
running	run	running
studies	studi	study
wolves	wolv	wolf
mice	mice	mouse

researchers research researcher

The generated files were:

outputs/stem_lemma_comparison.txt

and

outputs/stem_lemma_corpus_sample.txt

Observation

The results clearly showed that stemming simply removes word endings without considering grammar, while lemmatization attempts to return the correct dictionary form of a word. In several examples, lemmatization produced more meaningful results.

7. Task 4 – Part-of-Speech Tagging

Part-of-Speech tagging was performed using the spaCy library.

Each word in the sample sentences was assigned a grammatical category such as:

- Noun
- Verb
- Adjective
- Adverb
- Determiner
- Conjunction

The generated output was stored in:

outputs/pos_tagging_results.txt

Observation

The POS tagger correctly identified the grammatical role of most words. The output also grouped nouns, verbs, adjectives, and adverbs separately, making the results easier to understand.

8. Task 5 – Bag-of-Words Representation

The final task was to generate a Bag-of-Words representation.

The script created:

- A vocabulary of unique words
- Word frequency dictionaries
- A dense BoW matrix representing each sentence numerically

The generated output was saved in:

outputs/bow_results.txt

Observation

The Bag-of-Words model ignores word order and represents sentences only through word frequencies. Although simple, it provides a useful numerical representation that can be used for many traditional machine learning algorithms.

9. Challenges Faced

While running the project, a few required libraries and language resources were not installed on the local machine.

The following setup steps were completed successfully:

- Installed the NLTK library.
- Downloaded the required NLTK resources (punkt_tab and stopwords).
- Installed the spaCy library.
- Downloaded the en_core_web_sm language model.

After completing these setup steps, all scripts executed successfully and generated the expected output files.

10. Key Learnings

Through this project, I learned:

- The importance of preprocessing before applying NLP models.
- How different tokenization methods produce different outputs.
- The practical difference between stemming and lemmatization.
- How POS tagging identifies grammatical categories in text.
- How Bag-of-Words converts textual data into numerical features.
- How to install and configure NLP libraries required for Python projects.

11. Conclusion

The Day 1 tasks were completed successfully. The preprocessing pipeline executed without issues after installing the required dependencies, and all expected output files were generated. This project provided practical exposure to the fundamental stages of text preprocessing and

helped build a stronger understanding of how raw text is prepared for Natural Language Processing applications.